

GenAI: Test Execution

This module explains how **Generative AI and GitHub Copilot enhance the Test Execution phase** of STLC by accelerating script creation, execution, and result analysis.

Role of GenAI in Test Execution

Test Execution involves:

- Running test cases
- Recording results
- Reporting defects

Generative AI assists by:

- Generating execution scripts
 - Automating command sequences
 - Interpreting test results
 - Suggesting fixes
 - Optimizing execution order
-

How GenAI Supports Test Execution

Area	GenAI Contribution
Script generation	Creates run scripts
Parallel execution	Suggests configuration
Failure analysis	Explains errors
Retry logic	Generates code
Reporting	Creates summaries

Providing Scripts: Example

Scenario: Run PyTest Suite

Prompt

Create script to run pytest tests and generate report.

GenAI Output

```
pytest --html=report.html
```

Interpreting Test Results: Example

Sample Error

```
AssertionError: Expected 200 but got 500
```

GenAI Explanation

- API returned server error
 - Possible backend exception
 - Check logs
-

Demo: Optimizing Test Execution Using GitHub Copilot

Step 1

Create file `run_tests.sh`

```
# Script to run tests in parallel
```

Step 2

Copilot Suggests

```
pytest -n auto --maxfail=2 --disable-warnings
```

Step 3

Add Retry Logic

```
# Retry failed tests
```

Copilot Suggests

```
pytest --reruns 2
```

Benefits

- Faster execution
 - Reduced manual work
 - Clearer insights
-

Limitations

- May misinterpret logs
 - Needs validation
-

Quick Check

Q1. What is test execution?

Running test cases

Q2. True/False

GenAI can interpret errors.

True

Q3. Best practice?

Review AI output

Summary

Generative AI improves test execution by:

Automating scripts

Analyzing failures

Speeding execution

But **human oversight remains essential.**
